Оценка ролевой сегрегации в языковых моделях методом интроспекции мыслей

Даниил Привезенцев[1*]

[1*]Национальный исследовательский университет «Высшая школа экономики», Москва, Россия.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Аннотация

Рост числа агентных сценариев поверх больших языковых моделей делает стоимость инференса, объем контекста и надежность промежуточных решений центральными ограничителями практического применения AI-ассистентов в разработке программного обеспечения. В статье предлагается научно строгая переработка исходной постановки задачи оценки ролевой сегрегации на основе метода Introspection of Thought (INoT). В отличие от черновых версий работы, статья не выдает неподтвержденные архитектурные claims и не фабрикует экспериментальные результаты. Вклад работы состоит в четырех элементах: (i) аудите литературы с исключением слабых и непроверяемых источников; (ii) формализации гибридной агентной архитектуры, в которой роли *planner*, *worker*, *critic*, *self-check*, *memory/compression* и *policy controller* координируются вокруг внутреннего интроспективного цикла; (iii) введении системы метрик качества, токенной эффективности, стоимости, надежности и поддерживаемости кода, включая метрики *utility-per-token*, *quality-per-dollar*, *verified-code-rate* и *maintainability-aware success*; (iv) разработке воспроизводимого экспериментального протокола и синтетического, но воспроизводимого раздела по совокупной стоимости владения (TCO). Аналитически показано, что интроспективная архитектура выигрывает у классической внешне оркестрируемой мультиагентной схемы тогда, когда снижение повторной передачи контекста и уменьшение вероятности дорогого перезапуска превосходят стоимость внутренних раундов критики, компрессии и самопроверки.

Keywords: Большие языковые модели, AI agents, Introspection of Thought, Software Engineering, Token Efficiency, Economics of Inference, Code Reliability

1 Введение

Современная разработка программного обеспечения протекает в условиях постоянного переключения между кодовой базой, системой контроля версий, задачами, документацией и результатами автоматических проверок. Эмпирические исследования показывают, что такие переключения контекста заметно увеличивают когнитивную нагрузку и время возвращения к прерванной задаче [2–4]. На этом фоне ассистенты на основе больших языковых моделей (LLM) стали рассматриваться как средство частичной компенсации когнитивных издержек, однако переход от чат-модели к агентному контуру приводит к новой проблеме: растут не только функциональные возможности системы, но и суммарные токенные, временные и организационные издержки ее работы.

Классические LLM-агенты обычно реализуются как цикл «мышление–действие–наблюдение» с внешней оркестрацией, вызовами инструментов, делегированием ролям и повторным обращением к крупной модели на каждом значимом шаге [6, 17]. Такой дизайн особенно полезен при реальном использовании инструментов и изоляции привилегий, но он же создает повторную передачу длинного контекста между ролями, дублирование промежуточных артефактов и каскадное накопление стоимости. Для задач ревью, локального рефакторинга, документного анализа и обоснования изменений эта архитектура может оказываться избыточной.

Одним из возможных ответов на эту проблему является перенос части ролевого взаимодействия внутрь одного контекста модели. Идея тестового улучшения рассуждения через промежуточные шаги хорошо известна из работ по Chain-of-Thought, ReAct, Self-Refine и Reflexion [5–8]. Недавняя работа Sun и Zeng вводит *Introspection of Thought* (INoT) как схему, в которой одна модель имитирует несколько ролей и организует внутренний цикл генерации, критики и коррекции [1]. Однако сама по себе интроспекция не снимает методологических вопросов. Промежуточный текст нельзя автоматически трактовать как faithful explanation скрытого вычислительного процесса модели [9]. Кроме того, в литературе почти отсутствует строгая постановка задачи, связывающая внутреннюю ролевую сегрегацию именно с экономикой инференса, надежностью кода и поддерживаемостью артефактов в инженерных пайплайнах.

Центральная исследовательская проблема статьи состоит в следующем: может ли агентная архитектура с внутренней интроспекцией ролей повысить отношение полезности к токенам и к стоимости инференса без ухудшения качества и инженерной надежности по сравнению с одношаговым одноагентным режимом и классической мультиагентной схемой с внешней оркестрацией?

Вклад работы состоит в аудите исходной библиографии, формализации гибридной архитектуры агента с ролями *planner*, *worker*, *critic*, *self-check*, *memory/compression* и *policy controller*, построении аналитической модели стоимости, введении новых метрик эффективности и подготовке воспроизводимого протокола экспериментов и TCO-анализа.

2 Связанные работы и позиционирование

2.1 От пошаговых рассуждений к интроспективному тестовому улучшению

Работа Wei et al. показала, что явное порождение промежуточных шагов рассуждения может заметно улучшать решение сложных задач [5]. ReAct объединяет рассуждение и действие во внешней среде, позволяя чередовать текстовые шаги и вызовы инструментов [6]. Self-Refine переносит цикл генерации, обратной связи и исправления в рамки одной модели [7], а Reflexion добавляет использование языковой обратной связи и рефлексивной памяти в агентном контуре [8]. На этой линии Sun и Zeng предлагают INoT как более явно структурированный внутренний дебат ролей [1].

Для нашей постановки важны два вывода. Во-первых, тестовое улучшение качества без дообучения действительно возможно за счет промежуточной самокритики. Во-вторых, промежуточный текст не должен трактоваться как гарантированно верное объяснение вычислительного процесса модели. Поэтому в данной статье интроспекция мыслей рассматривается операционально: как управляемый механизм генерации кандидатов, критики и селективного перезапуска, а не как прямое наблюдение скрытого мышления модели.

2.2 Мультиагентные системы для программной инженерии

Классические работы Weiss и Wooldridge формируют базовую теоретическую рамку для мультиагентных систем [10, 11]. В прикладном LLM-контексте роль-специализированные системы получили широкое развитие в работах MetaGPT и ChatDev [12, 13]. Эти подходы показывают, что разделение ролей на архитектора, программиста и тестировщика может улучшать декомпозицию сложных задач, но одновременно создает накладные расходы на межагентную коммуникацию, синхронизацию и повторную передачу состояния.

Систематические обзоры фиксируют быстрый рост числа LLM-ориентированных мультиагентных систем и одновременно подчеркивают их проблемы: ошибка одной роли легко распространяется по цепочке, а стоимость координации возрастает с длиной взаимодействия [14, 15]. Для программной инженерии это особенно заметно в задачах, где несколько ролей читают одни и те же длинные фрагменты кода или логов. Параллельно benchmarks семейства SWE-bench и системы типа SWE-agent демонстрируют, что реальная автоматизация software engineering требует сочетания длинного контекста, редактирования файлов, вызовов тестов и повторных итераций [16, 17].

2.3 Компрессия контекста, самопроверка кода и поддерживаемость

Для длинных контекстов критично, что увеличение числа токенов повышает не только стоимость, но и риск информационного шума. LongLLMLingua и LLMLingua-2 показывают, что компрессия промпта может одновременно снижать задержку и поддерживать или даже улучшать качество на длинном контексте

[20, 21]. Для программной инженерии эта идея получила прямое развитие в работе Jia et al., где компрессия кодового контекста для issue resolution уменьшает токенный бюджет и улучшает результативность на SWE-bench [22]. Самопроверка кода не сводится к языковой рефлексии: compiler feedback, тесты и статический анализ должны оставаться внешними и наблюдаемыми сигналами [23–25].

2.4 Публично подтвержденные сведения о современных системах

Наряду с академической литературой необходимо аккуратно отделять публично подтвержденные факты о современных системах от догадок о скрытой внутренней архитектуре. Для статьи это особенно важно в отношении ‘DeepSeek‘, ‘GLM-4.5‘ и ‘OpenAI/ChatGPT‘.

Таблица 1 Публично подтвержденные и неподтвержденные claims о современных системах

Система	Подтверждено первичными источниками	Не подтверждено и не используется как факт
DeepSeek-V3	Технический отчет подтверждает архитектуру MoE, использование Multi-head Latent Attention и ориентацию на cost-efficient training/inference [26].	Публично не подтверждена внутренняя мультиагентная архитектура или использование INoT-подобного ролевого дебата.
DeepSeek-R1	Технический отчет подтверждает reasoning-oriented post-training и RL-схему для моделей семейства R1 [27].	Не подтверждено, что продуктовые агенты DeepSeek используют конкретную скрытую схему многоролевой оркестрации.
GLM-4.5	Публично подтверждены agentic/reasoning/coding-позиционирование модели и наличие technical report [28].	Не подтверждено, что GLM-4.5 реализует именно внутренний мультиагентный дебат как основную вычислительную схему.
OpenAI / ChatGPT agent	Официально подтверждены unified agentic system, браузер, терминал, коннекторы и публичный system card [29, 30]. OpenAI также публикует SDK-паттерны handoff и агентных workflow [31, 32].	Не подтверждена внутренняя роль-архитектура самого ChatGPT agent beyond public product description; поэтому любые детали о скрытом multi-agent decomposition не используются как факт.

2.5 Исследовательский зазор и научная новизна

Существующие работы покрывают либо внешнюю мультиагентную координацию, либо последовательные схемы саморефлексии одной модели. Недостаточно изученным остается промежуточный вариант: *гибридная агентная архитектура*, в

которой основное ролевое взаимодействие выполняется внутри одного контекста, а внешние специализированные процедуры используются только селективно для компрессии, проверки и управления инструментами. Научная новизна предлагаемого подхода состоит не в самом факте использования промежуточных рассуждений, а в их формализации как механизма управления экономикой инференса и инженерным качеством в agentic software engineering.

3 Научная проблема, гипотезы и формальная постановка

Пусть задана задача $x \in \mathcal{X}$, исходный контекст $\mathcal{C}_0$, множество разрешенных инструментов $\mathcal{Z}$ и ограничения по качеству, времени и стоимости. Агентная система должна построить ответ или патч y, максимизируя качество $Q(y, x)$ и одновременно минимизируя токенный расход T, денежную стоимость C и риск ненадежного изменения кода.

Мы рассматриваем три класса архитектур: **Single-Large**, **Classical-MAS** и **Hybrid-INoT**. Гипотеза **H1** состоит в том, что при длинном общем контексте Hybrid-INoT дает более высокий *utility-per-token*, чем Classical-MAS, за счет уменьшения повторной передачи контекста. Гипотеза **H2** утверждает, что малые вспомогательные модели для компрессии и самопроверки экономически оправданы, если снижают вероятность дорогого повторного вызова крупной модели сильнее, чем стоят сами. Гипотеза **H3** состоит в том, что преимущество Hybrid-INoT уменьшается на задачах, где действительно необходимы параллельные вызовы инструментов, изоляция привилегий или длительная внешняя среда исполнения.

4 Метод

4.1 Архитектура ролей и поток данных

Предлагаемая система использует роли *policy controller*, *memory/compression*, *planner*, *worker*, *critic* и *self-check*. Сначала *policy controller* очищает вход и проверяет допустимость инструментальных действий. Затем *memory/compression* извлекает релевантный контекст $\hat{\mathcal{C}} \subseteq \mathcal{C}_0$. После этого *planner* выбирает внутренний интроспективный режим или внешний агентный режим. Внутри интроспективного режима *worker* порождает K кандидатов, *critic* присваивает им оценки риска и полезности, а *self-check* либо принимает лучший кандидат, либо инициирует селективный повтор только для тех элементов, которые не прошли внешнюю проверку.

4.2 Формальные обозначения

Используем следующие обозначения:

$x \in \mathcal{X}$	задача,
$g(x) \in \mathcal{G}$	класс задачи,

$\mathcal{C}_0$	исходный контекст,
$\hat{\mathcal{C}}$	сжатый контекст,
K	число кандидатов,
I	максимальное число итераций,
p_i^L, r_i^L	входные и выходные токены крупной модели,
p_i^S, r_i^S	входные и выходные токены малой модели,
c_L^{in}, c_L^{out}	цены токена крупной модели,
c_S^{in}, c_S^{out}	цены токена малой модели,
ρ	вероятность дорогого повторного запуска,
Q	качество решения, нормированное в $[0, 1]$.

Общая токенная стоимость траектории определяется как

$$T = \sum_{i=1}^{I}(p_i^L + r_i^L) + \sum_{i=1}^{I}(p_i^S + r_i^S) + T_{\text{tool}}. \tag{1}$$

Для классической внешне оркестрируемой мультиагентной схемы:

$$T_{\text{MAS}} = \sum_{i=1}^{I}\sum_{a\in\mathcal{A}}(p_{i,a} + r_{i,a}) + D_{\text{ctx}} + H_{\text{coord}}, \tag{2}$$

где D_{ctx} — стоимость повторной передачи одних и тех же фрагментов контекста разным агентам, а H_{coord} — накладные расходы внешней координации.

Для предлагаемой Hybrid-INoT архитектуры:

$$T_{\text{INoT}} = T_{\text{core}} + T_{\text{cmp}} + T_{\text{check}} + T_{\text{rerun}}, \tag{3}$$

где T_{core} — основной внутренний раунд ролевого дебата, T_{cmp} — компрессия контекста, T_{check} — селективная проверка, T_{rerun} — стоимость повторных запусков только при провале верификации.

$$\mathbb{E}[T_{\text{MAS}} - T_{\text{INoT}}] > 0 \Longleftrightarrow \mathbb{E}[D_{\text{ctx}} + H_{\text{coord}}] > \mathbb{E}[T_{\text{cmp}} + T_{\text{check}} + T_{\text{rerun}} - T_{\text{core}}^{\text{saved}}], \tag{4}$$

4.3 Денежная стоимость и условие выгодности малых моделей

Денежная стоимость инференса задается как

$$C_{\text{inf}} = c_L^{in}\sum_i p_i^L + c_L^{out}\sum_i r_i^L + c_S^{in}\sum_i p_i^S + c_S^{out}\sum_i r_i^S. \tag{5}$$

Если малая модель используется только как фильтр самопроверки, ее применение выгодно тогда, когда она снижает ожидаемую стоимость повторного запуска крупной модели:

$$C^{S}_{\text{check}} < (\rho_0 - \rho_1)\, C^{L}_{\text{rerun}}, \tag{6}$$

где ρ_0 и ρ_1 — вероятности повторного запуска без и с малой самопроверкой соответственно, а C^{L}_{rerun} — ожидаемая стоимость такого повторного запуска.

4.4 Алгоритмы

Algorithm 1 Гибридный интроспективный цикл

Require: задача x, контекст $\mathcal{C}_0$, бюджет токенов B, максимум итераций I
Ensure: итоговое решение $y^\star$

```
C ← PolicySanitize(x, C_0)
Ĉ ← CompressIfNeeded(C)
m ← RouteMode(x, Ĉ)
if m = external then
    return ExternalAgentWorkflow(x, Ĉ)
end if
for i = 1 to I do
    π_i ← Planner(x, Ĉ, B)
    Y_i ← GenerateCandidates(π_i, K)
    S_i ← CriticScore(Y_i)
    y_i ← SelectBest(Y_i, S_i)
    v_i ← SelfCheck(y_i)
    if v_i = pass then
        return y_i
    else
        Ĉ ← UpdateContextWithFailures(Ĉ, v_i)
    end if
end for
return FallbackBestCandidate()
```

4.5 Система метрик

Для неоднородного набора задач вводится агрегированное качество

$$Q = \sum_{b=1}^{B} w_b Q_b, \qquad \sum_{b=1}^{B} w_b = 1, \tag{7}$$

где $Q_b \in [0, 1]$ — нормированное качество на бенчмарке или классе задач b, а w_b — заранее объявленные веса.

Algorithm 2 Селективный множественный подход с повторным запуском по необходимости

Require: набор кандидатов $\mathcal{Y} = \{y_1, \ldots, y_K\}$, оценки критика s_k, бюджет повторов R

Ensure: отфильтрованный кандидат $y^\star$

```
Отсортировать 𝒴 по убыванию s_k
Оставить верхние M ≤ K кандидатов
for каждый y_m из верхнего множества do
    q_m ← ExternalVerify(y_m)
    if q_m = pass then
        return y_m
    end if
end for
if R > 0 then
    Сформировать новый промпт только по выявленным ошибкам
    return SelectiveRerun(R − 1)
end if
return кандидат с максимальным s_k
```

Algorithm 3 Компрессия контекста для длинных задач

Require: контекст $\mathcal{C}_0$, порог длины τ_{len}

Ensure: контекст $\hat{\mathcal{C}}$

```
if |𝒞_0| ≤ τ_len then
    return 𝒞_0
end if
Разбить 𝒞_0 на смысловые блоки
Оценить блоки по релевантности к задаче и плотности проверяемых сигналов
Удалить дубликаты, низкоинформативные логи и вторичную документацию
Сохранить сигнатуры API, failing traces, изменяемые файлы и ограничения задачи
return Ĉ
```

Далее определяются основные метрики эффективности:

$$U_{\text{tok}} = \frac{Q}{T}, \tag{8}$$

$$Q_{\$} = \frac{Q}{C_{\text{inf}}}, \tag{9}$$

$$VCR = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}[\texttt{tests}_i = 1 \wedge \texttt{static}_i = 1 \wedge \texttt{security}_i = 1], \tag{10}$$

$$MAS = \frac{1}{N} \sum_{i=1}^{N} \mathit{Success}_i \cdot \mathit{Verified}_i \cdot M_i. \tag{11}$$

Algorithm 4 Самопроверка кода и оценка поддерживаемости

Require: патч или кодовый ответ y

Ensure: метка верификации v, оценка поддерживаемости M

$t \leftarrow \textsc{RunTests}(y)$
$s \leftarrow \textsc{RunStaticAnalysis}(y)$
$cc \leftarrow \textsc{CyclomaticComplexity}(y)$
$dup \leftarrow \textsc{DuplicationRate}(y)$
$warn \leftarrow \textsc{LintWarnings}(y)$
$doc \leftarrow \textsc{DocumentationCoverage}(y)$
$M \leftarrow w_{cc}(1 - \tilde{cc}) + w_{dup}(1 - \tilde{dup}) + w_{warn}(1 - \tilde{warn}) + w_{doc}\tilde{doc}$
if $t = \texttt{pass}$ and $s = \texttt{pass}$ **then**
    $v \leftarrow \texttt{pass}$
else
    $v \leftarrow \texttt{fail}$
end if
return (v, M)

Оценка поддерживаемости M_i вычисляется по нормализованным прокси-показателям:

$$M_i = w_{cc}(1 - \widetilde{CC}_i) + w_{dup}(1 - \widetilde{Dup}_i) + w_{warn}(1 - \widetilde{Warn}_i) + w_{doc}\widetilde{Doc}_i, \tag{12}$$

где $\widetilde{CC}_i$ — нормализованная цикломатическая сложность [34], $\widetilde{Dup}_i$ — степень дублирования, $\widetilde{Warn}_i$ — нормализованное число предупреждений статического анализа, $\widetilde{Doc}_i$ — нормализованное покрытие кодовой документации.

Для надежности агентного контура дополнительно используются:

$$SCR = \frac{\#\{\text{некорректные кандидаты, пойманные self-check}\}}{\#\{\text{все некорректные кандидаты}\}}, \tag{13}$$

$$RFR = \frac{\#\{\text{патчи без пост-хок rollback}\}}{\#\{\text{примененные патчи}\}}, \tag{14}$$

где SCR — recall самопроверки, а RFR — rollback-free rate.

5 Воспроизводимый экспериментальный протокол

5.1 Шкала длины запросов и классы задач

Чтобы оценка не зависела от одной произвольной длины контекста, вводится шкала:

$$\begin{aligned} short: &\quad |\mathcal{C}_0| \le 512, \\ medium: &\quad 512 < |\mathcal{C}_0| \le 2048, \\ long: &\quad 2048 < |\mathcal{C}_0| \le 8192, \end{aligned}$$

$$very\ long: \quad |\mathcal{C}_0| > 8192.$$

Задачи делятся на шесть классов: *single-step*, *multi-step*, *tool-using*, *coding*, *debugging*, *document reasoning*. Для каждого класса задается ожидаемое поведение архитектуры.

Таблица 2 Классы задач и ожидаемое поведение предлагаемой архитектуры

Класс задачи	Основной режим	Ожидаемое поведение
single-step	внутренний	Один кандидат, облегченный critic, без внешних инструментов; интроспекция минимальна, чтобы не проиграть одноагентному baseline по токенам.
multi-step	внутренний	Planner задает план, worker генерирует несколько гипотез, critic выявляет логические пропуски.
tool-using	гибридный	Policy controller определяет допустимые инструменты; self-check обязателен перед действием записи или исполнения.
coding	гибридный	Worker формирует патч, self-check запускает тесты, статический анализ и maintainability scoring.
debugging	гибридный	Сначала компрессия failing traces и релевантных файлов, затем несколько кандидатов исправления и селективный rerun.
document reasoning	внутренний	Memory/compression устраняет шум длинной документации; critic проверяет полноту и противоречия цитируемых фрагментов.

5.2 Наборы данных и synthetic-but-reproducible расширение

Базовый протокол включает три публичных набора:

1. **HumanEval** для коротких задач генерации кода [18];
2. **MBPP** для базовых программных задач естественного языка [19];
3. **SWE-bench** для задач исправления дефектов в реальных репозиториях [16].

Эти наборы дополняются двумя *synthetic but reproducible* семействами, потому что публичные benchmarks недостаточно покрывают длинный документный контекст и контролируемые tool-use сценарии:

1. **Synthetic Tool-Use Suite**: задачи генерируются из конечной грамматики действий над файлами, командной строкой и структурированными артефактами; для каждой задачи создается детерминированный oracle правильной последовательности.

2. **Synthetic Document Reasoning Suite**: из публично разрешенных технических текстов формируются задачи на сравнение версий, выявление противоречий, составление кратких решений и проверку соблюдения ограничений; генератор фиксируется seed-значениями и публикуется вместе с шаблонами.

5.3 Базовые конфигурации

Сравнение проводится по четырем архитектурным режимам:

1. **B0 Single-Large**: один вызов крупной модели, без критика и самопроверки;
2. **B1 Single-Large+Self-Refine**: одна крупная модель с последовательным циклом self-feedback [7];
3. **B2 Classical-MAS**: planner, worker, critic и verifier как отдельные агенты с внешней оркестрацией;
4. **B3 Hybrid-INoT**: предлагаемый подход с внутренней ролевой сегрегацией и селективными малыми помощниками.

Дополнительно задаются абляции: *no-critic*, *no-compression*, *no-self-check*, *large-model-only* и *always-external*. Они нужны, чтобы отделить вклад интроспекции от вклада компрессии, внешних проверок и маршрутизации.

5.4 Протокол запуска и статистика

Для стохастических режимов каждая конфигурация запускается не менее чем с пятью случайными seed-значениями на HumanEval, MBPP и synthetic suites. Для дорогих issue-resolution задач допускается не менее трех запусков при условии публикации конкретных seed-значений и полного списка задач. Отчет включает средние значения метрик и 95% доверительные интервалы, bootstrap-интервалы по задачам с не менее чем 10 000 ресэмплирований, парный критерий Мак-Немара для бинарных исходов успеха, критерий Вилкоксона для попарного сравнения токенов, стоимости и задержки, а также поправку Holm–Bonferroni при множественных сравнениях. Закрытые API-модели без фиксируемого snapshot ID выносятся в отдельный observational appendix и не смешиваются с воспроизводимым треком.

6 Экономический анализ и TCO

6.1 Формализация совокупной стоимости владения

Для месячного горизонта расчета совокупная стоимость владения определяется как

$$TCO_{\text{month}} = C_{\text{inf}} + C_{\text{hw}} + C_{\text{energy}} + C_{\text{storage}} + C_{\text{ops}}, \tag{15}$$

где

$$C_{\text{hw}} = \frac{CapEx_{\text{cpu}} + CapEx_{\text{gpu}} + CapEx_{\text{ram}}}{L_{\text{amort}}}, \tag{16}$$

$$C_{\text{energy}} = P_{\text{avg}} \cdot h_{\text{month}} \cdot c_{\text{kWh}}, \tag{17}$$

$$C_{\text{ops}} = C_{\text{monitoring}} + C_{\text{maintenance}} + C_{\text{human-review}}. \tag{18}$$

6.2 Synthetic-but-reproducible стендовый сценарий

Таблица 3 показывает *не эмпирически измеренный*, а синтетический воспроизводимый пример. Его задача — не заявить достигнутую экономию как факт, а показать, как именно сравнивать архитектуры при равных предпосылках.

Таблица 3 Синтетический, но воспроизводимый пример месячного TCO для стендового развертывания

Статья затрат / допущение	Classical-MAS	Hybrid-INoT
Число задач в месяц	10,000	10,000
Ожидаемые input токены крупной модели / задача	22,000	9,000
Ожидаемые output токены крупной модели / задача	3,200	1,800
Ожидаемые токены малых помощников / задача	0	1,800
Синтетическая стоимость инференса, USD / месяц	664	497
Амортизация CPU/RAM-сервера, USD / месяц	220	220
Амортизация GPU-ускорителей, USD / месяц	360	180
Storage и резервные артефакты, USD / месяц	20	20
Энергопотребление, USD / месяц	70	35
Сопровождение и human review, USD / месяц	300	300
Итого синтетический TCO, USD / месяц	1,634	1,252

Все значения в таблице 3 получены из заранее объявленных предпосылок: одна оркестрационная машина, одинаковый месячный workload, синтетические тарифы на токены и разные потребности в локальных ускорителях. При этих предпосылках Hybrid-INoT дает снижение TCO на 23.4%. Это не результат эксперимента на реальной системе, а воспроизводимый пример вычисления.

6.3 Когда подход выгоден и когда нет

Предлагаемая архитектура вероятнее всего экономически выгодна, если задачи содержат длинный общий контекст, который в Classical-MAS пришлось бы многократно пересылать независимым ролям, если значимая доля ошибок может быть поймана дешевой самопроверкой до полного перезапуска крупной модели, если параллельная работа ролей не является критичным источником ускорения, а малая модель или внешняя проверка достаточно надежны, чтобы уменьшать ρ в формуле (6). Подход может быть невыгоден при обратных условиях: очень короткие *single-step* задачи, высокий выигрыш от истинного параллелизма, слабая способность модели следовать ролевому формату, а также сценарии, где безопасность требует жесткой изоляции каждого инструмента и каждой привилегии в отдельном процессе.

7 Обсуждение

7.1 Почему интроспекция мыслей релевантна именно для AI agents

Для обычного одношагового чат-ответа промежуточная интроспекция может быть лишь способом улучшить финальный текст. Для AI agents значение выше: каждое промежуточное решение влияет на выбор инструмента, объем следующего контекста, число последующих итераций и, в конечном счете, на цену всей траектории. Следовательно, интроспекция здесь выполняет одновременно функцию управления качеством и функцию управления стоимостью.

7.2 Надежность кода, самопроверка и поддерживаемость

Ключевая инженерная идея статьи состоит в отказе от наивного допущения, что хороший текстовый ответ автоматически означает надежное изменение кода. Верификация должна быть внешней и наблюдаемой: тесты, компилятор, линтеры, статический анализ и security checks. Этот тезис согласуется с работами по compiler feedback и статическому анализу в кодогенерации [23, 24]. Поддерживаемость также рассматривается как первичная инженерная цель. Успешное решение, которое проходит тесты ценой резкого роста цикломатической сложности, дублирования или предупреждений анализа, не должно считаться полным успехом.

7.3 Ограничения метода и угрозы валидности

Предлагаемый подход имеет несколько существенных ограничений. Внутренний ролевой текст не гарантирует faithful explanation вычислительного процесса модели [9]. На задачах с подлинной независимостью ролей внешняя мультиагентная архитектура может быть предпочтительнее. Компрессия контекста сама по себе может удалить редкий, но критичный факт, если селектор релевантности спроектирован неудачно. Метрики поддерживаемости зависят от выбранного набора статических прокси и не заменяют полноценное архитектурное ревью. Наконец, закрытые API-модели подвержены дрейфу версии; без фиксированных snapshot ID они не подходят для строгого воспроизводимого трека.

8 Заключение

В статье предложена научно состоятельная переработка постановки задачи оценки ролевой сегрегации в языковых моделях методом интроспекции мыслей. Новая версия опирается только на реальные и цитируемые академические либо официальные первичные источники, не использует неподтвержденные архитектурные claims как установленный факт, формализует ролевую интроспекцию как механизм управления качеством, токенами, стоимостью и надежностью, а также вводит воспроизводимый экспериментальный протокол и синтетический TCO-расчет с явными допущениями.

References

[1] H. Sun and S. Zeng, "Introspection of Thought Helps AI Agents," arXiv preprint arXiv:2507.08664, 2025.

[2] A. J. Ko, R. DeLine, and G. Venolia, "Information Needs in Collocated Software Development Teams," in *Proceedings of the 29th International Conference on Software Engineering*, 2007, pp. 344–353.

[3] C. Parnin and S. Rugaber, "Resumption Strategies for Interrupted Programming Tasks," *Software Quality Journal*, vol. 19, no. 1, pp. 5–34, 2011.

[4] M. K. Goncalves, C. R. B. de Souza, and V. M. Gonzalez, "Collaboration, Information Seeking and Communication: An Observational Study of Software Developers' Work Practices," *Journal of Universal Computer Science*, vol. 17, no. 11, pp. 1529–1550, 2011.

[5] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824–24837.

[6] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv preprint arXiv:2210.03629, 2022.

[7] A. Madaan *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," arXiv preprint arXiv:2303.17651, 2023.

[8] N. Shinn *et al.*, "Reflexion: Language Agents with Verbal Reinforcement Learning," arXiv preprint arXiv:2303.11366, 2023.

[9] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting," arXiv preprint arXiv:2305.04388, 2023.

[10] G. Weiss, *Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence*. Cambridge, MA, USA: MIT Press, 1999.

[11] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, UK: Wiley, 2009.

[12] S. Hong *et al.*, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," arXiv preprint arXiv:2308.00352, 2023.

[13] C. Qian *et al.*, "ChatDev: Communicative Agents for Software Development," arXiv preprint arXiv:2307.07924, 2023.

[14] T. Guo *et al.*, "Large Language Model based Multi-Agents: A Survey of Progress and Challenges," arXiv preprint arXiv:2402.01680, 2024.

[15] C. Qian *et al.*, "Scaling Large Language Model-based Multi-Agent Collaboration," arXiv preprint arXiv:2406.07155, 2024.

[16] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," arXiv preprint arXiv:2310.06770, 2023.

[17] J. Yang *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," arXiv preprint arXiv:2405.15793, 2024.

[18] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021.

[19] J. Austin *et al.*, "Program Synthesis with Large Language Models," arXiv preprint arXiv:2108.07732, 2021.

[20] H. Jiang *et al.*, "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression," arXiv preprint arXiv:2310.06839, 2023.

[21] Z. Pan *et al.*, "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression," arXiv preprint arXiv:2403.12968, 2024.

[22] H. Jia, E. T. Barr, and S. Mechtaev, "Compressing Code Context for LLM-based Issue Resolution," arXiv preprint arXiv:2603.28119, 2026.

[23] Z. Bi *et al.*, "Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback," arXiv preprint arXiv:2403.16792, 2024.

[24] C.-Y. Su and C. McMillan, "Do Code LLMs Do Static Analysis?," arXiv preprint arXiv:2505.12118, 2025.

[25] M. Sepidband, H. Taherkhani, S. Wang, and H. Hemmati, "Enhancing LLM-Based Code Generation with Complexity Metrics: A Feedback-Driven Approach," arXiv preprint arXiv:2505.23953, 2025.

[26] DeepSeek-AI *et al.*, "DeepSeek-V3 Technical Report," arXiv preprint arXiv:2412.19437, 2024.

[27] DeepSeek-AI *et al.*, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," arXiv preprint arXiv:2501.12948, 2025.

[28] Z.ai, "GLM-4.5: Agentic, Reasoning, and Coding Foundation Models," arXiv preprint arXiv:2508.06471, 2025.

[29] OpenAI, "Introducing ChatGPT agent: bridging research and action," Jul. 17, 2025. [Online]. Available: https://openai.com/index/introducing-chatgpt-agent/

[30] OpenAI, "ChatGPT agent System Card," Jul. 17, 2025. [Online]. Available: https://openai.com/index/chatgpt-agent-system-card/

[31] OpenAI, "Agents SDK," 2026. [Online]. Available: https://platform.openai.com/docs/guides/agents-sdk/

[32] OpenAI, "A Practical Guide to Building Agents," 2026. [Online]. Available: https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf

[33] ISO/IEC 25010:2023, "Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model," International Organization for Standardization, 2023.

[34] T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.